

CSE-214 Online - 3

Behavioural Design Pattern

Duration: 30 Minutes

Problem: E-commerce Return and Refund Workflow

An e-commerce platform needs to manage the lifecycle of a **product return request**. During processing, a return request may go through several different conditions. The operations that can be performed on the request depend on its current condition.

Return Workflow

- **Requested:** When a return request is created, it starts in the Requested condition. The return reason may be updated multiple times. The request may also be approved, rejected, or cancelled.
- **Approved:** The return reason can no longer be modified. The request may still be cancelled if the returned item has not yet been delivered. The returned item may be marked as delivered.
- **Delivered:** The request can no longer be cancelled. The item may be inspected. If it satisfies the return policy, refund processing begins; otherwise, the request becomes rejected.
- **Processing Refund:** The return reason cannot be modified and the request cannot be cancelled. The refund may succeed or fail. If it fails, it may be attempted again later.
- **Refunded / Rejected / Cancelled:** These are final conditions. No further operation should change the request.

Operations

Your system should provide suitable operations for the following actions:

```
updateReason(String reason)
approve()
reject()
cancel()
itemDelivered()
inspect(boolean eligible)
refundSuccessful()
refundFailed()
```

For any operation that is not valid at a particular point in the workflow, the system should produce an appropriate message rather than performing the operation.

Task

Design and implement the return/refund management system according to the requirements above.

1. Correctly handle the behaviour of each operation throughout the return lifecycle.
2. Perform the appropriate transitions when an operation succeeds.
3. Prevent operations that are not valid at a particular point in the workflow.
4. Keep the implementation modular so that behaviour associated with one part of the workflow can be changed without significantly affecting the others.
5. Allow new return-processing conditions or behaviours to be incorporated later with minimal modification to existing code.

Note: You may introduce any necessary classes, interfaces, and methods to produce a clean object-oriented solution.